

クラスはオブジェクトの設計図^{せっけいず}

I. クラスとコンストラクタ

「ユーザー定義^{ていぎ}オブジェクト」が作れるようになると、次にぶつかる壁^{かべ}が「同じ形^{おな かたち}のオブジェクトを100人分^{にんぶん}作りたいときはどうするの？」という問題^{もんだい}です。

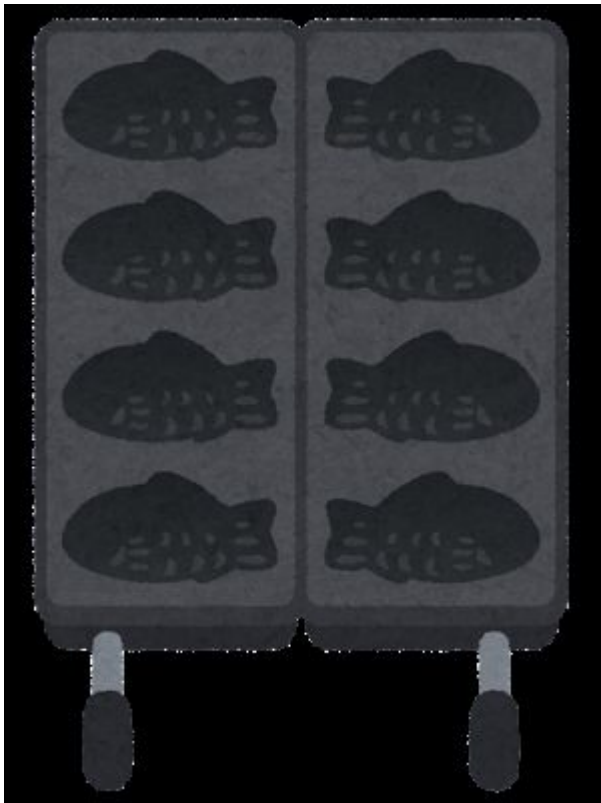
ひとり^{ひとり}ずつ手書き^{て が}でオブジェクトを作るのは大変^{たいへん}ですね。そこで登場^{とうじょう}するのが「クラス (Class)」と「コンストラクタ (Constructor)」です。

これを「たい焼き^やの型^{かた}」に例^{たと}えて説明^{せつめい}します。

1. クラス (Class) とは？

クラスは、オブジェクトを作るための「設計図^{せっけいず} (せっけいず)」または「型^{かた} (かた)」のことです。

- クラス：たい焼き^やの「型^{かた}」。これ自体^{じたい}は食べられませんが、形^{かたち}を決めています。



- オブジェクト (インスタンス)：型^{かた}から焼^やかれた「たい焼き^や」。実際^{じっさい}に食べられる実物^{じつぶつ}です。



クラスを一度作っておけば、同じ特徴を持ったオブジェクトを何度でも、一瞬で作ることができます。

[!note] 「インスタンス」という言葉は、「オブジェクト」とほぼ同じ意味ですが、より具体的なイメージです。例えば、「オブジェクト」が「たい焼き」全般を指すすると、「インスタンス」は、今、あなた食べようとしている目の前の「たい焼き」という具体的な存在を指すイメージです。

2. コンストラクタ (Constructor) とは？

コンストラクタは、クラスの中に書く「初期化（しょきか）のための魔法」です。

たい焼きを焼くときに、「中身は何にする？」「あんこ？ クリーム？」と決める作業のことです。オブジェクトが誕生する瞬間に、名前や年齢などのデータをセットする役割を持っています。

3. コードで見てみよう！

クラスという「型」を使って、2人の学生を作ってみましょう。

```
// 1. クラス（設計図）を作る
class Student {
  // 2. コンストラクタ（中身をセットする役割）
  constructor(name, age) {
    this.name = name; // この人の名前は～
    this.age = age;   // この人の年齢は～
  }

  // 3. メソッド（この型で作られた人ができる動（うご）き）
  sayHello() {
    console.log(`こんにちは、${this.name}です！`);
  }
}

// 4. クラスを使って実物（インスタンス）を作る
const studentA = new Student("アリ", 20);
const studentB = new Student("リン", 22);

studentA.sayHello(); // "こんにちは、アリです！"
studentB.sayHello(); // "こんにちは、リンです！"
```

[!note] `this.name = name;`の右の`name`は引数として渡された`name`で、左の`this.name`は、`student`というクラスのインスタンスが内部に持つデータ、つまりオブジェクトのプロパティです。
`studentA.name` や `studentA[name]` で"アリ"を取り出すことができます。

4. 例え話：ハンコとスタンプ

クラスは『ハンコ』です。
ハンコの形（名前や年齢の枠）は決まっているよね。
実際に紙にポン！と押して出てきた図形が『オブジェクト（インスタンス）』。
押すたびにインクの色（データ）を変えれば、同じ形だけど少し違うスタンプがたくさん作れるでしょ？
`new`という言葉を使ってハンコを押すイメージなんですよ！

5. 大切なキーワードまとめ

ことば 言葉	い み 意味	たと ばなし 例え話
Class（クラス）	オブジェクトの設計図	たい焼きの型 / ハンコ
Constructor（コンストラクタ）	最初にデータをセットする場所	中身を詰める作業
Instance（インスタンス）	クラスから作られた実物	焼けたたい焼き / 押されたスタンプ
<code>new</code>	新しいインスタンスを作る 命令	「焼いて！」という注文

 学生へのアドバイス

最初は `this.name = name` のような書き方にびっくりするかもしれないけど、これは『新しく生まれる自分（this）のなまえを、もらった名前にセットしてね』という意味なんです。
慣れてくると、100人でも1000人でも同じキャラクターを簡単に作れるようになるから、すごく便利ですよ！

いかがでしょうか？「型（クラス）」を作って、「注文（new）」して、「実物（インスタンス）」を作る。この流れが分かれば、大規模なプログラムも怖くありません！

II. 継 承：効率的に新しいものを作る

「クラス」という設計図の作り方がわかったら、次はもっと便利な「継 承（けいしょう / Inheritance）」という仕組みを覚えましょう！

継 承とは、例えて言えば、「親のいいところを受け継いで、新しい独自の特徴をプラスすること」です。

1. 継 承（Inheritance）とは？

「^{けいしょう}継承」とは、すでに作ってあるクラス（設計図）をベースにして、^{あた}新しいクラスを作ることです。

たとえば、「^{どうぶつ}動物（Animal）」というクラスがあれば、それを^{けいしょう}継承して「^{いぬ}イヌ（Dog）」や「^{ねこ}ネコ（Cat）」を作ることができます。

- 「^{どうぶつ}動物」なら誰でも持っている^{とくちょう}特徴（^{なまえ}名前、^た食べる^{うご}動き）は、^{そのま}そのままろう。
- 「^{いぬ}イヌ」だけの^{とくちょう}特徴（^は吠える^{うご}動き）は、^{あた}新しく^か書き^た足す。

これが^{けいしょう}継承のすごいところです！

2. ^か書き^{かた}方のキーワード：^{extends} と ^{super}

JavaScriptで^{けいしょう}継承を使うときには、2つの^{だいじ}大事な^{ことば}言葉を使います。

1. ^{extends}（^いイクステンズ）：「^{ひろ}～を広げる（^{けいしょう}継承する）」という^い意味。
2. ^{super}（^{すー}スーパー）：「^{おや}親（^{おや}親クラス）」という^い意味。親の^{おや}コンストラクタを^よ呼び^だ出すときに^{つか}使います。

```
// 1. 親クラス（ベース）：人間
class Person {
  constructor(name) {
    this.name = name;
  }
  sayHello() {
    console.log(`こんにちは、${this.name}です。`);
  }
}

// 2. 子クラス（継承（けいしょう））：学生
// 「Person（人間）」をベースに「InternationalStudent」を作る
class InternationalStudent extends Person {
  constructor(name, country) {
    super(name); // 親クラスのコンストラクタに「名前」を渡す（重要！）
    this.country = country; // 新しい特徴（国籍）をプラス
  }

  // 3. 新しい動（うご）きをプラス
  introduce() {
    console.log(`${this.country}から来ました。よろしくお願いします！`);
  }
}

const user = new InternationalStudent("アハメド", "エジプト");
user.sayHello(); // 親からもらった動（うご）き
user.introduce(); // 自分だけの新しい動（うご）き
```

3. ^{がくせい}学生への^{たと}例え話：^{ばなし}スマホの^{しん}進化

最初の携帯電話（親クラス）は、『電話をする』『メールを送る』機能を持っていました。
次に登場したスマートフォン（子クラス）は、携帯電話の機能を継承しているから、もちろん電話もメールもできます。
でも、そこに『アプリを入れる』とか『顔認証（かおにんしょう）』という新しい機能を追加したんだ。これが『継承』のイメージです！

4. なぜ継承を使うの？

継承のメリットをリストアップしてみましょう。

- **楽ができる**：同じことを何度も書かなくていいから、コードが短くなる。
- **間違いが減る**：基本のルール（親）を直せば、全員（子）に反映されるから便利。

まとめテーブル

ことば 言葉	よ 読み方	やくわり 役割
Parent Class	おや 親クラス	きほん せつけいず 基本の設計図（ベース）
Child Class	こ 子クラス	けいしょう あたら つく せつけいず 継承して新しく作る設計図
extends	イクステンズ	おや う つ せんげん 「親から受け継ぐよ！」という宣言
super()	スーパー	おや うご あいず 「親のコンストラクタを動かして！」という合図

【!TIP】「オーバーライド」もできます！オブジェクトの「オーバーライド」と同様に、クラスでも、親からもらった動きを、子クラスで書き換える「オーバーライド（上書き）」が使えます。「親のやり方より、僕のやり方のほうが得意だよ！」と上書きするイメージだと教えてあげると、より理解が深まります。

いかがでしょうか？「基本の形をもらって、自分流にアレンジする」。これが継承のパワーです！

IV. まとめ：プログラマーのように考えよう

オブジェクト指向（しこう）は、単なる書き方ではなく、「複雑（ふくざつ）な現実（げんじつ）を整理（せいり）する方法」です。

アクションプラン

https://developer.mozilla.org/ja/docs/Learn_web_development/Core/Scripting/A_first_splash

上の「数当てゲーム」を、オブジェクトを使って次のように改良（かいりょう）を考えてみてください。

- 「ゲーム全体」をひとつの大きなオブジェクトにする。
- プレイヤーの「これまでの予想履歴（よそうりれき）」をプロパティに保存する。
- 「当たり・ハズレのはんてい」をメソッドにする。

難しい用語も、RPGや自己紹介カードのように身近なものに例えれば、きっと理解（りかい）できるはずで
す。オブジェクトを使（つか）って、よりスマートでプロフェッショナルなプログラムを作（つく）っていきましょう。自信
（じしん）を持って挑（も）戦（ちょうせん）してください！